

Informe eSports Arena Manager

Caso semestral - Desarrollo FullStack I (Backend)

Integrantes: - Sebastián Ahumada
- Benjamín Gutiérrez
- Emanuel Serey

Grupo: 1

Asignatura: Desarrollo Fullstack 1

Sección: 002D

Repositorio:

<https://github.com/Emanuel-Serey/proyecto-esports-arena-manager>

1. Introducción

El presente informe describe el desarrollo del proyecto eSports Arena Manager, una plataforma backend orientada a la gestión de torneos competitivos de videojuegos. El sistema permite administrar usuarios, juegos, equipos, torneos, inscripciones, sanciones, partidas, resultados, rankings y premios, utilizando una arquitectura basada en microservicios con Spring Boot.

2. Objetivo del proyecto

El objetivo del proyecto fue implementar una solución backend distribuida mediante microservicios independientes, cada uno con su propia responsabilidad, base de datos H2, operaciones CRUD y comunicación REST con otros servicios cuando era necesario validar información.

3. Tecnologías utilizadas

- Java 25
- Spring Boot
- Spring Web
- Spring Data JPA
- H2 Database
- OpenFeign
- Lombok
- Maven
- Postman
- Git y GitHub
- Trello
- IntelliJ IDEA

4. Arquitectura general del sistema

El sistema eSports Arena Manager fue desarrollado bajo una arquitectura de microservicios, donde cada servicio cumple una responsabilidad específica dentro del flujo de una competencia eSports. Esta arquitectura permite separar el sistema en componentes independientes, facilitando el desarrollo, mantenimiento, pruebas y escalabilidad del proyecto.

Cada microservicio cuenta con su propia estructura Spring Boot, organizada en capas como Controller, Service, Repository, DTO, Model y manejo de excepciones. Además, cada servicio posee su propia base de datos H2, evitando una base de datos única centralizada.

La comunicación entre microservicios se realiza mediante API REST y clientes OpenFeign. Esto permite que un servicio consulte información de otro cuando necesita validar datos externos. Por ejemplo, tournament-service consulta a game-service para verificar que el juego asociado a un torneo exista o esté disponible antes de crear el registro.

Internamente, cada microservicio sigue el siguiente flujo:

Cliente/Postman → Controller → ServiceImpl → Repository → H2 → Respuesta JSON

El Controller recibe las solicitudes HTTP y las deriva al Service. El ServiceImpl contiene la lógica de negocio, las validaciones y, cuando corresponde, la comunicación con otros microservicios. El Repository se encarga de acceder a la base de datos mediante JPA, mientras que H2 almacena la información propia de cada microservicio.

Esta arquitectura permite mantener una separación clara de responsabilidades, mejorar la trazabilidad del sistema y facilitar futuras mejoras o ampliaciones del proyecto.

5. Microservicios implementados

El sistema fue dividido en diez microservicios principales, cada uno encargado de una parte específica del dominio del proyecto. Esta división permite que cada servicio administre su propia información y exponga sus endpoints REST de forma independiente.

Microservicio	Puerto	Responsabilidad
user-service	8087	Gestiona usuarios del sistema
game-service	8083	Gestiona juegos disponibles
tournament-service	8084	Gestiona torneos
team-service	8085	Gestiona equipos y miembros
registration-service	8086	Gestiona inscripciones a torneos
sanction-service	8088	Gestiona sanciones de usuarios y equipos
match-service	8089	Gestiona partidas del torneo
result-service	8090	Gestiona resultados de partidas
ranking-service	8091	Gestiona ranking y posiciones
prize-service	8092	Gestiona premios del torneo

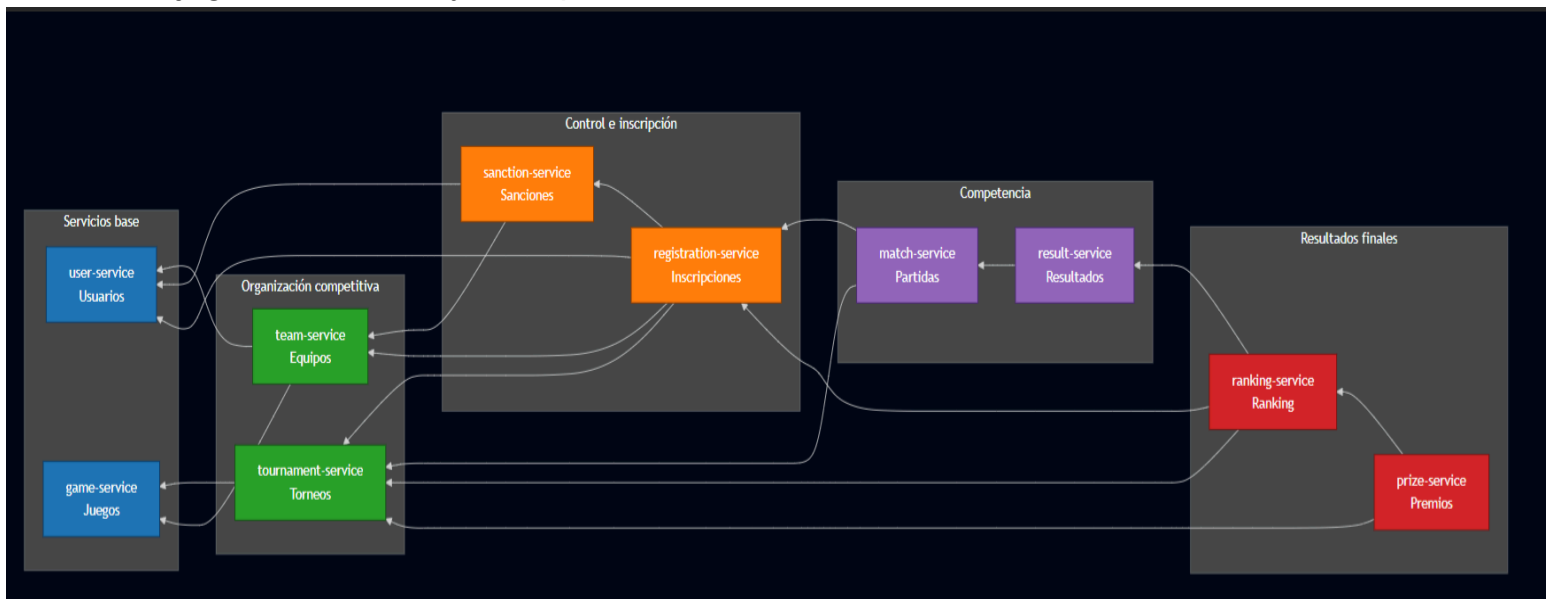
Cada microservicio cuenta con su propio CRUD o conjunto de operaciones principales, permitiendo crear, consultar, actualizar o eliminar información según la responsabilidad que cumple dentro del sistema.

6. Diagrama del ecosistema de microservicios

El ecosistema de microservicios representa cómo se relacionan los distintos servicios dentro del sistema eSports Arena Manager. Las flechas del diagrama indican que un microservicio de origen consulta o consume información de otro microservicio de destino mediante OpenFeign.

Por ejemplo, registration-service necesita consultar información de tournament-service, team-service, user-service y sanction-service para validar correctamente una inscripción. De esta forma, antes de registrar una

participación, el sistema puede comprobar si el torneo existe, si el equipo o jugador es válido y si no posee sanciones activas.



7. Comunicación entre microservicios

La comunicación entre microservicios se realiza mediante llamadas REST utilizando OpenFeign. Esto permite que un servicio pueda consultar información de otro sin acceder directamente a su base de datos.

Cada microservicio mantiene su propia base de datos H2, por lo que las relaciones entre servicios no se implementan mediante claves foráneas físicas entre bases de datos distintas. En su lugar, se utilizan IDs lógicos y validaciones realizadas mediante API REST.

Microservicio	Consume
tournament-service	game-service
team-service	user-service, game-service
sanction-service	user-service, team-service
registration-service	tournament-service, team-service, user-service, sanction-service
match-service	tournament-service, registration-service
result-service	match-service
ranking-service	tournament-service, registration-service, result-service
prize-service	tournament-service, ranking-service

Esta comunicación permite mantener la independencia de cada microservicio, pero al mismo tiempo conservar la consistencia de la información dentro del flujo general del sistema.

8. Modelo de datos

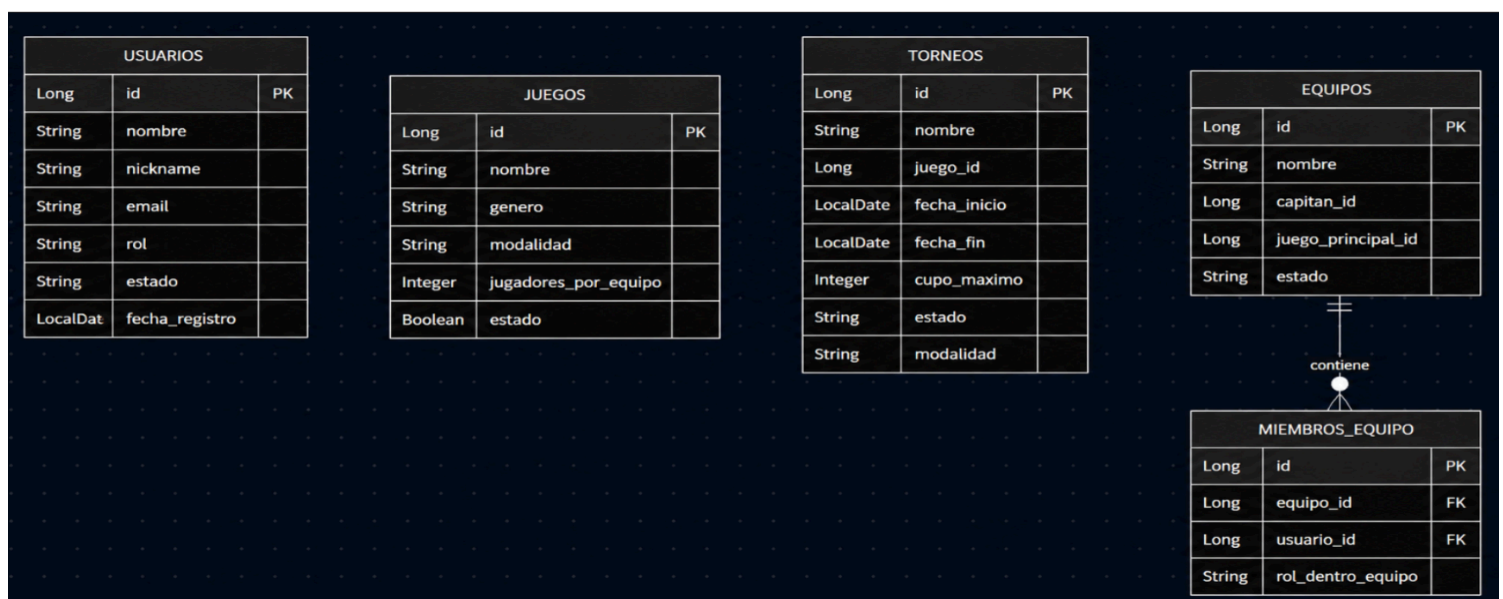
Cada microservicio administra su propia base de datos H2, siguiendo el principio de persistencia independiente por servicio. Esto significa que cada microservicio almacena únicamente la información que corresponde a su responsabilidad.

Las relaciones entre microservicios se manejan mediante identificadores lógicos, como `usuario_id`, `juego_id`, `torneo_id`, `equipo_id` o `participante_id`. Estos identificadores permiten asociar información entre servicios sin crear dependencias físicas directas entre distintas bases de datos.

8.1 Modelo relacional: usuarios, juegos, torneos y equipos

En esta parte del modelo se encuentran las entidades base del sistema. USUARIOS almacena los participantes registrados, JUEGOS define los videojuegos disponibles para torneos, TORNEOS administra las competencias y EQUIPOS gestiona los equipos participantes.

Además, la tabla MIEMBROS_EQUIPO permite representar la relación entre un equipo y sus integrantes. Un equipo puede tener varios miembros, por lo que se utiliza una relación de uno a muchos entre EQUIPOS y MIEMBROS_EQUIPO.



8.2 Modelo relacional: inscripciones, sanciones y partidas

En esta sección se representan las entidades asociadas al control de participación en los torneos. INSCRIPCIONES almacena la relación lógica entre un torneo y sus participantes, utilizando campos como torneo_id, equipo_id o jugador_id.

La entidad SANCIONES permite registrar restricciones asociadas a usuarios o equipos. Estas sanciones pueden ser consultadas por otros servicios, por ejemplo, para evitar que un participante sancionado pueda inscribirse en un torneo.

Por otro lado, PARTIDAS representa los enfrentamientos dentro de un torneo, utilizando identificadores de participantes para indicar quiénes se enfrentan en cada partida.

INSCRIPCIONES		
Long	id	PK
Long	torneo_id	
Long	equipo_id	
Long	jugador_id	
String	tipo_participante	
String	estado	
LocalDate	fecha_inscripcion	

SANCIONES		
Long	id	PK
Long	usuario_id	
Long	equipo_id	
String	motivo	
LocalDate	fecha_inicio	
LocalDate	fecha_fin	
String	estado	
String	severidad	

PARTIDAS		
Long	id	PK
Long	torneo_id	
Long	participante_a_id	
Long	participante_b_id	
String	ronda	
LocalDateTime	fecha_hora	
String	estado	

8.3 Modelo relacional: resultados, rankings y premios

La última parte del modelo considera las entidades relacionadas con el cierre del flujo competitivo. RESULTADOS registra el ganador y los puntajes de cada partida. Luego, RANKINGS almacena la posición, puntos, victorias, derrotas y diferencia de cada participante dentro de un torneo.

Finalmente, PREMIOS permite asignar recompensas según el torneo, participante y posición obtenida. Esta información depende del estado final del ranking, por lo que prize-service puede consultar a ranking-service para validar la asignación correspondiente.

RESULTADOS		
Long	id	PK
Long	partida_id	
Long	ganador_id	
Integer	puntaje_a	
Integer	puntaje_b	
String	estado_validacion	
LocalDate	fecha_registro	

RANKINGS		
Long	id	PK
Long	torneo_id	
Long	participante_id	
Integer	puntos	
Integer	victorias	
Integer	derrotas	
Integer	diferencia	
Integer	posicion	

PREMIOS		
Long	id	PK
Long	torneo_id	
Long	participante_id	
Integer	posicion	
String	tipo_premio	
String	descripcion	
String	estado_entrega	
LocalDate	fecha_asignacion	

9. Endpoints principales

user-service

Método	Endpoint	Descripción
POST	<code>/api/usuarios</code>	Crear usuario
GET	<code>/api/usuarios</code>	Listar usuarios
GET	<code>/api/usuarios/{id}</code>	Buscar usuario por ID
PUT	<code>/api/usuarios/{id}</code>	Actualizar usuario
DELETE	<code>/api/usuarios/{id}</code>	Desactivar usuario
GET	<code>/api/usuarios/rol/{rol}</code>	Listar por rol
GET	<code>/api/usuarios/estado/{estado}</code>	Listar por estado
GET	<code>/api/usuarios/nickname/{nickname}</code>	Buscar por nickname

game-service

Método	Endpoint	Descripción
POST	/api/juegos	Crear juego
GET	/api/juegos	Listar juegos
GET	/api/juegos/{id}	Buscar juego por ID
GET	/api/juegos/activos	Listar juegos activos
PUT	/api/juegos/{id}	Actualizar juego
DELETE	/api/juegos/{id}	Desactivar juego

tournament-service

Método	Endpoint	Descripción
POST	/api/torneos	Crear torneo
GET	/api/torneos	Listar torneos
GET	/api/torneos/{id}	Buscar torneo por ID
PUT	/api/torneos/{id}	Actualizar torneo
PUT	/api/torneos/{id}/cancelar	Cancelar torneo
PUT	/api/torneos/{id}/cerrar	Cerrar torneo
GET	/api/torneos/juego/{juegoId}	Listar por juego
GET	/api/torneos/estado/{estado}	Listar por estado

team-service

Método	Endpoint	Descripción
POST	/api/equipos	Crear equipo
GET	/api/equipos	Listar equipos
GET	/api/equipos/{id}	Buscar equipo por ID
PUT	/api/equipos/{id}	Actualizar equipo
DELETE	/api/equipos/{id}	Desactivar equipo
POST	/api/equipos/{equipoId}/miembros	Agregar miembro
GET	/api/equipos/{equipoId}/miembros	Listar miembros
DELETE	/api/equipos/{equipoId}/miembros/{miembroId}	Eliminar miembro

registration-service

Método	Endpoint	Descripción
POST	/api/inscripciones	Crear inscripción
GET	/api/inscripciones	Listar inscripciones
GET	/api/inscripciones/{id}	Buscar inscripción por ID
PUT	/api/inscripciones/{id}/estado/{estado}	Actualizar estado
PUT	/api/inscripciones/{id}/cancelar	Cancelar inscripción
GET	/api/inscripciones/torneo/{torneoId}	Listar por torneo
GET	/api/inscripciones/equipo/{equipoId}	Listar por equipo
GET	/api/inscripciones/jugador/{jugadorId}	Listar por jugador

sanction-service

Método	Endpoint	Descripción
POST	/api/sanciones	Crear sanción
GET	/api/sanciones	Listar sanciones
GET	/api/sanciones/{id}	Buscar sanción por ID
PUT	/api/sanciones/{id}	Actualizar sanción
PUT	/api/sanciones/{id}/cerrar	Cerrar sanción
GET	/api/sanciones/usuario/{usuarioId}	Listar por usuario
GET	/api/sanciones/equipo/{equipoId}	Listar por equipo
GET	/api/sanciones/usuario/{usuarioId}/activa	Validar sanción activa de usuario
GET	/api/sanciones/equipo/{equipoId}/activa	Validar sanción activa de equipo

match-service

Método	Endpoint	Descripción
POST	/api/partidas	Crear partida
GET	/api/partidas	Listar partidas
GET	/api/partidas/{id}	Buscar partida por ID
PUT	/api/partidas/{id}	Actualizar partida
PUT	/api/partidas/{id}/estado/{estado}	Actualizar estado
PUT	/api/partidas/{id}/cancelar	Cancelar partida
GET	/api/partidas/torneo/{torneoId}	Listar por torneo

result-service

Método	Endpoint	Descripción
POST	/api/resultados	Crear resultado
GET	/api/resultados	Listar resultados
GET	/api/resultados/{id}	Buscar resultado por ID
PUT	/api/resultados/{id}	Actualizar resultado
PUT	/api/resultados/{id}/validar	Validar resultado
PUT	/api/resultados/{id}/anular	Anular resultado
GET	/api/resultados/partida/{partidaId}	Buscar por partida

ranking-service

Método	Endpoint	Descripción
POST	/api/rankings	Crear ranking
GET	/api/rankings	Listar rankings
GET	/api/rankings/{id}	Buscar ranking por ID
GET	/api/rankings/torneo/{torneoId}	Listar ranking por torneo
GET	/api/rankings/torneo/{torneoId}/participante/{participanteId}	Buscar posición de participante
PUT	/api/rankings/{id}	Actualizar ranking
DELETE	/api/rankings/torneo/{torneoId}/reiniciar	Reiniciar ranking
PUT	/api/rankings/resultado/{resultadoId}	Actualizar ranking por resultado

prize-service

Método	Endpoint	Descripción
POST	/api/premios	Asignar premio
GET	/api/premios	Listar premios
GET	/api/premios/{id}	Buscar premio por ID
GET	/api/premios/torneo/{torneoId}	Listar premios por torneo
GET	/api/premios/participante/{participanteId}	Listar premios por participante
PUT	/api/premios/{id}/entregar	Entregar premio
PUT	/api/premios/{id}/anular	Anular premio

10. Colección Postman

Las pruebas del sistema fueron realizadas mediante Postman, validando los principales flujos REST del proyecto. Estas pruebas permitieron comprobar la creación de datos base, la comunicación entre microservicios, las validaciones de negocio y el cierre del flujo competitivo.

1) Crear usuario

http://localhost:8087/api/usuarios

POST http://localhost:8087/api/usuarios

Docs Params Authorization Headers 8 Body Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "nombre": "Juan Perez",
3   "nickname": "juanp",
4   "email": "juanp@gmail.com",
5   "rol": "JUGADOR",
6   "estado": "ACTIVO"
7 }
```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "nombre": "Juan Perez",
4   "nickname": "juanp",
5   "email": "juanp@gmail.com",
6   "rol": "JUGADOR",
7   "estado": "ACTIVO",
8   "fechaRegistro": "2026-05-22"
9 }
```

2) Crear usuario 2

POST http://localhost:8087/api/usuarios

Docs Params Authorization Headers 8 Body Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "nombre": "Pedro Soto",
3   "nickname": "pedros",
4   "email": "pedros@gmail.com",
5   "rol": "JUGADOR",
6   "estado": "ACTIVO"
7 }
```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON Preview Visualize

```
1 {
2   "id": 2,
3   "nombre": "Pedro Soto",
4   "nickname": "pedros",
5   "email": "pedros@gmail.com",
6   "rol": "JUGADOR",
7   "estado": "ACTIVO",
8   "fechaRegistro": "2026-05-22"
9 }
```

3) Crear Juego

The screenshot shows a REST client interface with a POST request to `http://localhost:8083/api/juegos`. The request body is a JSON object representing a game. The response is a 201 Created status with a JSON object containing the game's details, including an assigned ID.

Request:

```
POST http://localhost:8083/api/juegos
{
  "nombre": "League of Legends",
  "genero": "MOBA",
  "modalidad": "EQUIPO",
  "jugadoresPorEquipo": 5,
  "estado": true
}
```

Response:

```
201 Created
{
  "id": 1,
  "nombre": "League of Legends",
  "genero": "MOBA",
  "modalidad": "EQUIPO",
  "jugadoresPorEquipo": 5,
  "estado": true
}
```

4) Crear Torneo

The screenshot shows a REST client interface with a POST request to `http://localhost:8084/api/torneos`. The request body is a JSON object representing a tournament. The response is a 201 Created status with a JSON object containing the tournament's details, including an assigned ID.

Request:

```
POST http://localhost:8084/api/torneos
{
  "nombre": "Torneo Apertura eSports",
  "juegoId": 1,
  "fechaInicio": "2026-06-01",
  "fechaFin": "2026-06-10",
  "cupoMaximo": 16,
  "estado": "ABIERTO",
  "modalidad": "EQUIPO"
}
```

Response:

```
201 Created
{
  "id": 2,
  "nombre": "Torneo Apertura eSports",
  "juegoId": 1,
  "fechaInicio": "2026-06-01",
  "fechaFin": "2026-06-10",
  "cupoMaximo": 16,
  "estado": "ABIERTO",
  "modalidad": "EQUIPO"
}
```

5) Crear Equipo

POST http://localhost:8085/api/equipos

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "nombre": "Team Alpha",
3   "capitanId": 1,
4   "juegoPrincipalId": 1,
5   "estado": "ACTIVO",
6   "miembros": [
7     {
8       "usuarioId": 1,
9       "rolDentroEquipo": "CAPITAN"
10    },
11    {
12      "usuarioId": 2,
13      "rolDentroEquipo": "JUGADOR"
14    }
15  ]
16 }
```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "nombre": "Team Alpha",
4   "capitanId": 1,
5   "juegoPrincipalId": 1,
6   "estado": "ACTIVO",
7   "miembros": [
8     {
9       "id": 1,
10      "usuarioId": 1,
11      "rolDentroEquipo": "CAPITAN"
12    },
13    {
14      "id": 2,
15      "usuarioId": 2,
16      "rolDentroEquipo": "JUGADOR"
17    }
18  ]
19 }
```

6) Crear Equipo 2

POST http://localhost:8085/api/equipos

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "nombre": "Team Beta",
3   "capitanId": 3,
4   "juegoPrincipalId": 1,
5   "estado": "ACTIVO",
6   "miembros": [
7     {
8       "usuarioId": 3,
9       "rolDentroEquipo": "CAPITAN"
10    },
11    {
12      "usuarioId": 4,
13      "rolDentroEquipo": "JUGADOR"
14    },
15    {
16      "usuarioId": 5,
17      "rolDentroEquipo": "JUGADOR"
18    }
19  ]
20 }
```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON Preview Visualize

```
1 {
2   "id": 2,
3   "nombre": "Team Beta",
4   "capitanId": 3,
5   "juegoPrincipalId": 1,
6   "estado": "ACTIVO",
7   "miembros": [
8     {
9       "id": 3,
10      "usuarioId": 3,
11      "rolDentroEquipo": "CAPITAN"
12    },
13    {
14      "id": 4,
15      "usuarioId": 4,
16      "rolDentroEquipo": "JUGADOR"
17    },
18    {
19      "id": 5,
20      "usuarioId": 5,
21      "rolDentroEquipo": "JUGADOR"
22    }
23  ]
24 }
```


7) Crear Sanción a usuario 1

POST http://localhost:8088/api/sanciones

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "usuarioId": 1,
3   "equipoId": null,
4   "motivo": "Conducta antideportiva",
5   "fechaInicio": "2026-05-01",
6   "fechaFin": "2026-05-10",
7   "estado": "ACTIVA",
8   "severidad": "MEDIA"
9 }
```

Body Cookies Headers 5 Test Results 201 Created

{ JSON Preview Visualize

```
1 {
2   "id": 2,
3   "usuarioId": 1,
4   "equipoId": null,
5   "motivo": "Conducta antideportiva",
6   "fechaInicio": "2026-05-01",
7   "fechaFin": "2026-05-10",
8   "estado": "ACTIVA",
9   "severidad": "MEDIA"
10 }
```

8) Intentar inscripción con jugador sancionado

POST http://localhost:8086/api/inscripciones

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "torneoId": 1,
3   "equipoId": null,
4   "jugadorId": 1,
5   "tipoParticipante": "JUGADOR",
6   "estado": "PENDIENTE"
7 }
```

Body Cookies Headers 4 Test Results 400 Bad Request

{ JSON Preview Debug with AI

```
1 {
2   "error": "No se puede inscribir un jugador con sanción activa"
3 }
```

9) Crear inscripción válida de equipo

POST http://localhost:8086/api/inscripciones

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "torneoId": 1,
3   "equipoId": 2,
4   "jugadorId": null,
5   "tipoParticipante": "EQUIPO",
6   "estado": "PENDIENTE"
7 }
```

Body Cookies Headers 5 Test Results 201 Created

{ JSON Preview Visualize

```
1 {
2   "id": 1,
3   "torneoId": 1,
4   "equipoId": 2,
5   "jugadorId": null,
6   "tipoParticipante": "EQUIPO",
7   "estado": "PENDIENTE",
8   "fechaInscripcion": "2026-05-22"
9 }
```

10) Listar inscripciones por Torneo

GET

http://localhost:8086/api/inscripciones/torneo/1

Docs

Params

Authorization

Headers 6

Body

Scripts

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

Body

Cookies

Headers 5

Test Results

200 OK

JSON

Preview

Visualize

1

2

3

4

5

6

7

8

9

10

11

{

{id: 1,

"torneoId": 1,

"equipoId": 2,

"jugadorId": null,

"tipoParticipante": "EQUIPO",

"estado": "PENDIENTE",

"fechaInscripcion": "2026-06-22"

}

11) Error al crear partida con participante no inscrito

POST

http://localhost:8089/api/partidas

Docs

Params

Authorization

Headers 8

Body

Scripts

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

2

3

4

5

6

7

8

{

"torneoId": 1,

"participanteAId": 1,

"participanteBId": 2,

"ronda": "Ronda 1",

"fechaHora": "2026-06-02T18:00:00",

"estado": "PROGRAMADA"

}

Body

Cookies

Headers 4

Test Results

400 Bad Request

JSON

Preview

Debug with AI

1

2

3

{

"error": "El participante no existe o no est\u00e1 inscrito"

}

12) Crear Partida

POST

http://localhost:8089/api/partidas

Docs

Params

Authorization

Headers 8

Body

Scripts

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

2

3

4

5

6

7

8

{

"torneoId": 1,

"participanteAId": 1,

"participanteBId": 2,

"ronda": "Ronda 1",

"fechaHora": "2026-06-02T18:00:00",

"estado": "PROGRAMADA"

}

Body

Cookies

Headers 5

Test Results

201 Created

JSON

Preview

Visualize

1

2

3

4

5

6

7

8

9

{

"id": 1,

"torneoId": 1,

"participanteAId": 1,

"participanteBId": 2,

"ronda": "RONDA 1",

"fechaHora": "2026-06-02T18:00:00",

"estado": "PROGRAMADA"

}

13) Crear Resultado

POST http://localhost:8090/api/resultados

Body

```
1 {
2   "partidaId": 1,
3   "ganadorId": 1,
4   "puntajeA": 2,
5   "puntajeB": 1,
6   "fechaRegistro": "2026-06-02"
7 }
```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "partidaId": 1,
4   "ganadorId": 1,
5   "puntajeA": 2,
6   "puntajeB": 1,
7   "estadoValidacion": "PENDIENTE",
8   "fechaRegistro": "2026-06-22"
9 }
```

14) Validar Resultado

PUT http://localhost:8090/api/resultados/1/validar

Body

```
1 {
2   "partidaId": 1,
3   "ganadorId": 1,
4   "puntajeA": 2,
5   "puntajeB": 1,
6   "estadoValidacion": "VALIDADO",
7   "fechaRegistro": "2026-06-02"
8 }
```

Body Cookies Headers 5 Test Results 200 OK

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "partidaId": 1,
4   "ganadorId": 1,
5   "puntajeA": 2,
6   "puntajeB": 1,
7   "estadoValidacion": "VALIDADO",
8   "fechaRegistro": "2026-06-22"
9 }
```

15) Actualizar Ranking por resultado

The screenshot shows a REST client interface with a PUT request to the URL `http://localhost:8091/apl/rankings/resultado/1?torneoId=1&participanteAId=1&participanteBId=2`. The request body is set to 'raw' and contains the number '1'. The response status is '200 OK'. The response body is shown in JSON format:

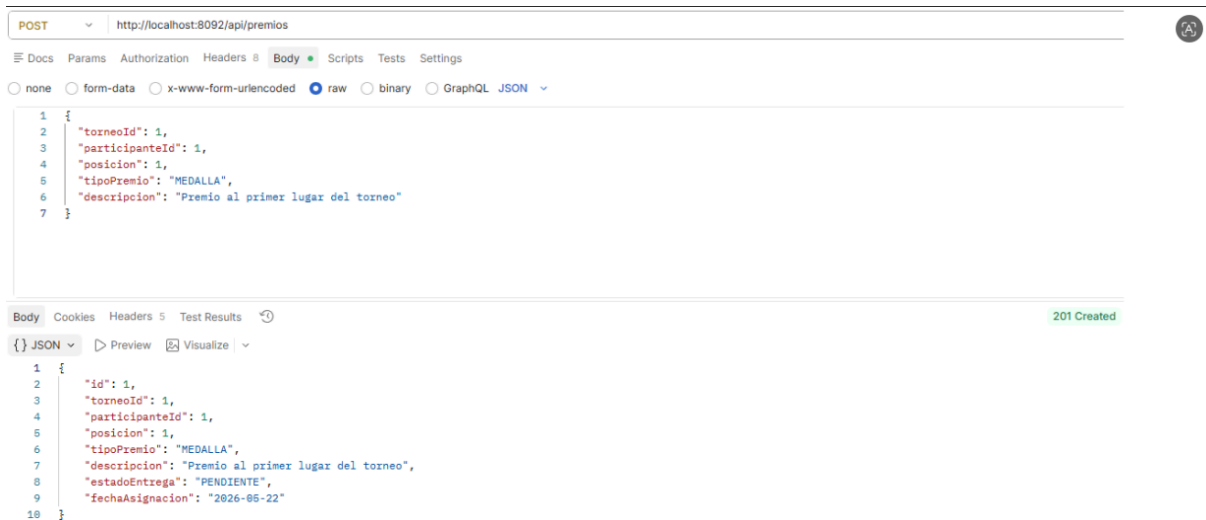
```
1 {
2   "id": 1,
3   "torneoId": 1,
4   "participanteId": 1,
5   "puntos": 3,
6   "victorias": 1,
7   "derrotas": 0,
8   "diferencia": 1,
9   "posicion": 1
10 }
```

16) Consultar Ranking por Torneo

The screenshot shows a REST client interface with a GET request to the URL `http://localhost:8091/apl/rankings/torneo/1`. The request has no body. The response status is '200 OK'. The response body is shown in JSON format:

```
1 [
2   {
3     "id": 1,
4     "torneoId": 1,
5     "participanteId": 1,
6     "puntos": 3,
7     "victorias": 1,
8     "derrotas": 0,
9     "diferencia": 1,
10    "posicion": 1
11  },
12  {
13    "id": 2,
14    "torneoId": 1,
15    "participanteId": 2,
16    "puntos": 0,
17    "victorias": 0,
18    "derrotas": 1,
19    "diferencia": -1,
20    "posicion": 2
21  }
22 ]
```

17) Asignar Premio

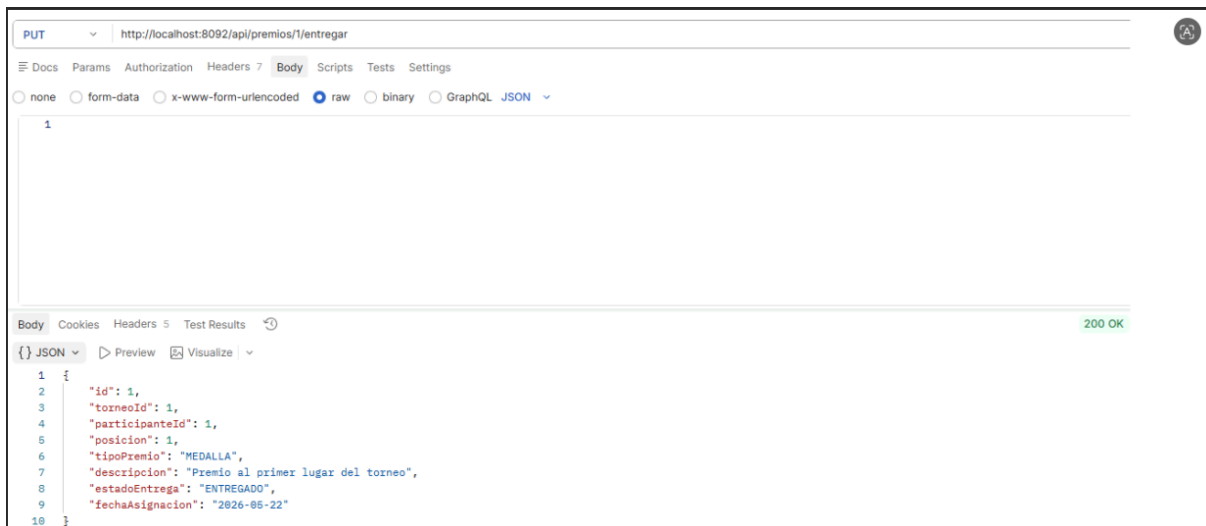


```
POST http://localhost:8092/api/premios

{
  "torneoId": 1,
  "participanteId": 1,
  "posicion": 1,
  "tipoPremio": "MEDALLA",
  "descripcion": "Premio al primer lugar del torneo"
}
```

```
{
  "id": 1,
  "torneoId": 1,
  "participanteId": 1,
  "posicion": 1,
  "tipoPremio": "MEDALLA",
  "descripcion": "Premio al primer lugar del torneo",
  "estadoEntrega": "PENDIENTE",
  "fechaAsignacion": "2026-05-22"
}
```

18) Marcar premio como entregado



```
PUT http://localhost:8092/api/premios/1/entregar
```

```
{
  "id": 1,
  "torneoId": 1,
  "participanteId": 1,
  "posicion": 1,
  "tipoPremio": "MEDALLA",
  "descripcion": "Premio al primer lugar del torneo",
  "estadoEntrega": "ENTREGADO",
  "fechaAsignacion": "2026-05-22"
}
```

11. Trabajo colaborativo

El trabajo colaborativo se organizó mediante GitHub, utilizando una rama principal main, una rama de integración development y ramas específicas para el desarrollo de cada microservicio. Esta estructura permitió separar el trabajo por responsabilidades, mantener trazabilidad de los cambios y facilitar la integración progresiva del proyecto.

Branches							New branch
Overview Yours Active Stale All Q Search branches...							
Branch	Updated	Check status	Behind	Ahead	Pull request		
development	2 days ago		0	0			...
team-service	4 days ago		53	0			...
tournament-service	4 days ago		58	0			...
ranking-service	5 days ago		18	0			...
prize-service	5 days ago		11	0			...
result-service	5 days ago		26	0			...
correccion-match-service	last week		36	0			...
correccion-registration-user	2 weeks ago		37	0			...
correccion-estructura-servicios	2 weeks ago		38	0			...
registration-service	2 weeks ago		39	0			...
match-service	2 weeks ago		43	0			...
sanction-service	2 weeks ago		47	0			...
game-service	2 weeks ago		69	0			...
user-service	2 weeks ago		75	0			...

En la evidencia se observa el uso de ramas específicas para organizar el desarrollo del proyecto. Se trabajó con una rama principal main, una rama de integración development y ramas independientes para cada microservicio, como user-service, game-service, tournament-service, team-service, registration-service, sanction-service, match-service, result-service, ranking-service y prize-service.

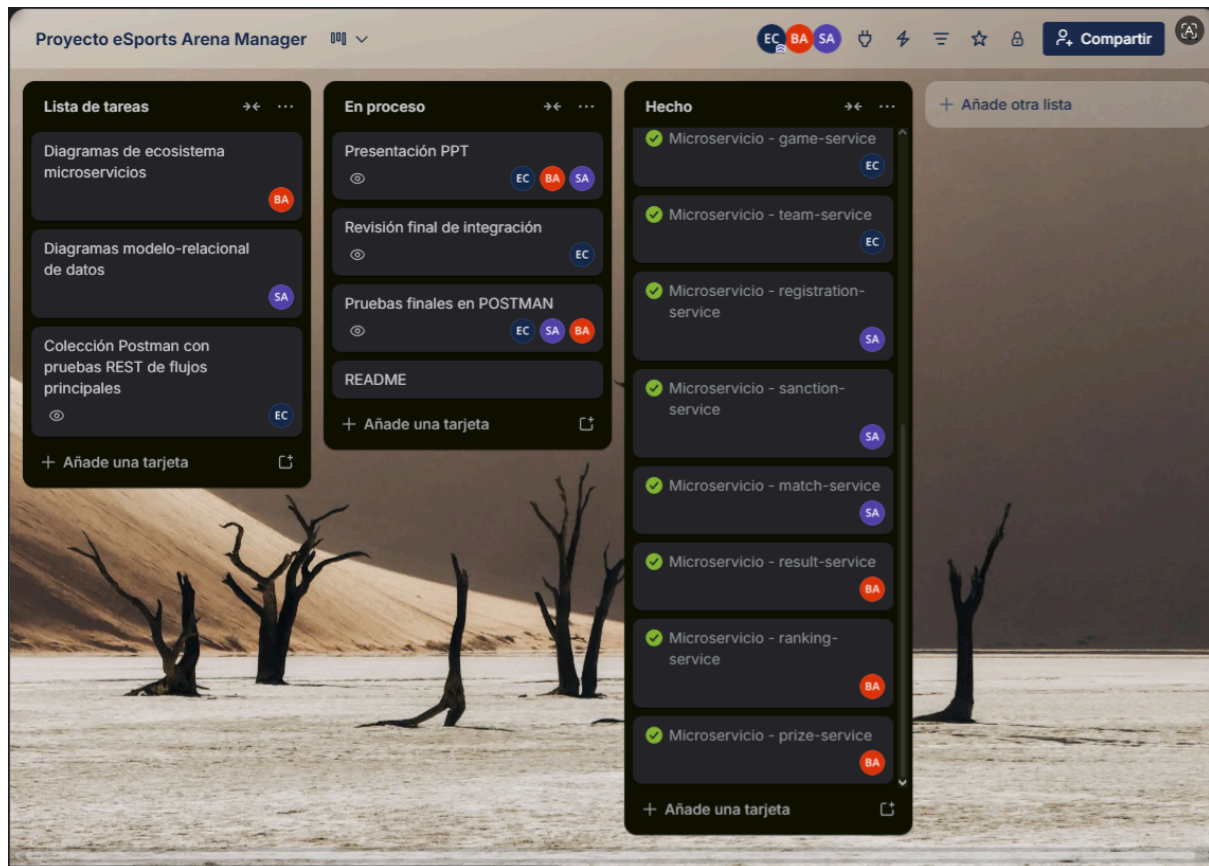
Además, se utilizaron ramas de corrección, como correccion-match-service, correccion-registration-user y correccion-estructura-servicios, para ajustar errores específicos antes de integrar los cambios al proyecto principal.

Planificación mediante Trello

También se utilizó Trello para organizar las tareas del equipo durante el desarrollo del proyecto. El tablero permitió dividir el trabajo en tareas

pendientes, tareas en proceso y tareas completadas, asignando responsables a cada actividad.

En Trello se registraron actividades relacionadas con el desarrollo de microservicios, elaboración de diagramas, pruebas en Postman, revisión de integración, README y presentación final.



En la evidencia se observa la distribución de tareas del equipo, incluyendo actividades de documentación, pruebas, revisión final y microservicios completados, junto con los responsables asignados a cada tarea.